- The conditions explicitly verified include a valid signature, valid recipient (worker), and sufficient balance in the escrow. If verification of these conditions is successful, the claim amount is transferred, and the smart contract and the micropayment channel are closed via the self-destruct statement. Imagine a bank getting imploded after every single channel is closed! This is impossible for a bank but possible with a smart contract.

Review this new paradigm by examining the smart contract code in the next listing.

Listing 7.1 MPC.sol

```
contract MPC{
    address payable public sender;
    address payable public recipient;
    constructor (address payable reciever) public payable
    {
        sender = msg.sender;                                              ❶
        recipient = receiver;                                             ❶
    }

    function isValidSignature(uint256 amount, bytes memory signedMessage)
        internal view returns (bool)
    {
        bytes32 message = prefixed(keccak256(abi.encodePacked(this,
                                                        amount)));
        return recoverSigner(message, signedMessage) == sender;           ❷
    }

    function claimPayment(uint256 amount, bytes memory signedMessage)
                                                    public{               ❸
        require(msg.sender == recipient,'Not a recipient');
        require(isValidSignature(amount, signedMessage),'Signature
                                                        Unmatch');
        require(address(this).balance > amount,'Insufficient Funds');     ❹
        recipient.transfer(amount);
        selfdestruct(sender);                                             ❺
    }

    function splitSignature(bytes memory sig) internal pure               ❺
        returns (uint8 v, bytes32 r, bytes32 s)
    {
        require(sig.length == 65,'Signature length');
        assembly{
```